

Sample Docker Compose + Docker Swarm project

What this delivers - A complete sample project that runs with either `docker-compose` (local/dev) or `docker stack deploy` (Docker Swarm). - Services: **front** (Next.js v15 App Router) on port **3000**, **back** (NestJS + MongoDB + Redis) on port **4000**, **mongo**, **redis**, and **nginx** which routes requests for `front.sample.ir`, `back.sample.ir` and serves a custom default HTML. - A **News** CRUD API implemented in the NestJS backend (module: `news`) with simple Redis caching for the list endpoint. - A lightweight Next.js frontend with pages & components that consume the API and demonstrate create/read/update/delete. - `Dockerfile`s for front/back/nginx, `docker-compose.yml` for local dev, and `docker-stack.yml` for Swarm deploy. - `.env` examples for both front and back.

Prerequisites

- Docker Engine (latest) and docker-compose (v2 CLI) installed.
 - For Swarm: initialize swarm (`docker swarm init`).
 - Node (only needed if you want to run locally without Docker). Not necessary for Docker flow.
 - Edit `/etc/hosts` (or your host resolver) to map the test domains used below to the Swarm manager / host IP (example added in instructions).
-

Project structure (top-level)

```
sample-project/
├ back/           # NestJS backend
├ front/          # Next.js front-end (App Router)
├ nginx/          # nginx reverse proxy + default html
├ docker-compose.yml # development compose (build)
├ docker-stack.yml  # swarm stack (uses pre-built images)
└ README.md        # this file
```

Quick usage (developer-friendly)

1) Local (docker-compose) - development/test

```
# From project root
# 1) create env files (examples are under each service folder). Edit if you need to.
# 2) Build and run everything locally
docker compose up --build
```

```
# open http://front.sample.ir:80 or http://localhost:3000 (if not using nginx)
# API: http://back.sample.ir:80 or http://localhost:4000
```

Note: `docker compose` (v2) uses the local bridge network. The compose file below maps ports so you can test on localhost.

2) Docker Swarm (single-node) - production-like

```
# Build images and tag them (replace tag with your repo/tag if you push to a
registry)
docker build -t sample_front:latest ./front
docker build -t sample_back:latest ./back
docker build -t sample_nginx:latest ./nginx

# Initialize swarm if not already:
docker swarm init

# Deploy stack (this uses docker-stack.yml that references the images above)
docker stack deploy -c docker-stack.yml sample

# Check services:
docker stack services sample

# Visit: http://front.sample.ir and http://back.sample.ir (map hosts to manager
IP as described below)
```

Important host mapping (for local testing)

Add to `/etc/hosts` (on Linux/macOS) or `C:\Windows\System32\drivers\etc\hosts` on Windows:

```
127.0.0.1 front.sample.ir
127.0.0.1 back.sample.ir
```

If your Docker Swarm manager is on a remote IP, use that IP instead of `127.0.0.1`.

Files (complete contents)

Below are the files to create. Copy them exactly into the paths shown. After the file listing you'll find explanations and run commands.

Root `docker-compose.yml` (local/dev)

```
version: '3.8'
services:
  mongo:
    image: mongo:6.0
    restart: unless-stopped
    volumes:
      - mongo_data:/data/db
    networks:
      - sample_net

  redis:
    image: redis:7
    restart: unless-stopped
    volumes:
      - redis_data:/data
    networks:
      - sample_net

  back:
    build: ./back
    env_file: ./back/.env
    ports:
      - "4000:4000"
    depends_on:
      - mongo
      - redis
    volumes:
      - ./back:/usr/src/app
    networks:
      - sample_net

  front:
    build: ./front
    env_file: ./front/.env
    ports:
      - "3000:3000"
    depends_on:
      - back
    volumes:
      - ./front:/usr/src/app
    networks:
      - sample_net

  nginx:
    build: ./nginx
```

```

    ports:
      - "80:80"
    depends_on:
      - front
      - back
    networks:
      - sample_net

volumes:
  mongo_data:
  redis_data:

networks:
  sample_net:
    driver: bridge

```

`docker-stack.yml` (for `docker stack deploy`) - uses pre-built images

```

version: '3.8'
services:
  mongo:
    image: mongo:6.0
    volumes:
      - mongo_data:/data/db
    networks:
      - sample_net

  redis:
    image: redis:7
    networks:
      - sample_net

  back:
    image: sample_back:latest
    env_file: ./back/.env
    ports:
      - target: 4000
        published: 4000
        protocol: tcp
        mode: host
    deploy:
      replicas: 1
      restart_policy:
        condition: on-failure
    networks:

```

```

    - sample_net

front:
  image: sample_front:latest
  env_file: ./front/.env
  ports:
    - target: 3000
      published: 3000
      protocol: tcp
      mode: host
  deploy:
    replicas: 1
    restart_policy:
      condition: on-failure
  networks:
    - sample_net

nginx:
  image: sample_nginx:latest
  ports:
    - target: 80
      published: 80
      protocol: tcp
      mode: host
  deploy:
    replicas: 1
  networks:
    - sample_net

volumes:
  mongo_data:
  redis_data:

networks:
  sample_net:
    driver: overlay

```

Note: `mode: host` in published ports makes services available directly on the node's IP/port (single-node swarm simplicity). For multi-node deploys you might use routing mesh (remove `mode: host`).

nginx/ files

nginx/Dockerfile

```
FROM nginx:alpine
COPY default.conf /etc/nginx/conf.d/default.conf
COPY html /usr/share/nginx/html
```

nginx/default.conf

```
upstream front_up {
    server front:3000;
}

upstream back_up {
    server back:4000;
}

server {
    listen 80;
    server_name front.sample.ir;

    location / {
        proxy_pass http://front_up;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
    }
}

server {
    listen 80;
    server_name back.sample.ir;

    location / {
        proxy_pass http://back_up;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

```
server {
    listen 80 default_server;
    server_name _;
    root /usr/share/nginx/html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

nginx/html/index.html (custom default page)

```
<!doctype html>
<html>
  <head>
    <meta charset="utf-8" />
    <title>Sample Docker Swarm Project</title>
  </head>
  <body>
    <h1>Sample Docker Swarm Project</h1>
    <p>This is the nginx default page. Use <a href="http://
front.sample.ir">front.sample.ir</a> or <a href="http://
back.sample.ir">back.sample.ir</a>.</p>
  </body>
</html>
```

back/ (NestJS) - files

This is a compact NestJS app that implements the News CRUD.

back/package.json

```
{
  "name": "sample-back",
  "version": "1.0.0",
  "scripts": {
    "start": "node dist/main.js",
    "start:dev": "nest start --watch",
    "build": "tsc -p tsconfig.build.json"
  },
  "dependencies": {
    "@nestjs/common": "^10.0.0",
    "@nestjs/core": "^10.0.0",
```

```

    "@nestjs/mongoose": "^10.0.0",
    "mongoose": "^7.0.0",
    "ioredis": "^5.3.2",
    "class-validator": "^0.14.0",
    "class-transformer": "^0.5.1",
    "reflect-metadata": "^0.1.13",
    "rxjs": "^7.8.0"
  },
  "devDependencies": {
    "@nestjs/cli": "^10.0.0",
    "typescript": "^5.2.2",
    "@types/node": "^20.4.2",
    "ts-node": "^10.9.1"
  }
}

```

back/tsconfig.json

```

{
  "compilerOptions": {
    "module": "commonjs",
    "declaration": false,
    "removeComments": true,
    "emitDecoratorMetadata": true,
    "experimentalDecorators": true,
    "target": "es2020",
    "sourceMap": true,
    "outDir": "dist",
    "skipLibCheck": true
  },
  "include": ["src/**/*.ts"]
}

```

back/tsconfig.build.json

```

{
  "extends": "../tsconfig.json",
  "exclude": ["node_modules", "test", "dist"]
}

```

back/.env (example)

```

MONGO_URI=mongodb://mongo:27017/newsdb
REDIS_HOST=redis

```



```
REDIS_PORT=6379  
PORT=4000
```

back/Dockerfile

```
FROM node:20-alpine AS builder  
WORKDIR /usr/src/app  
COPY package*.json tsconfig*.json ./  
COPY src ./src  
RUN npm ci --silent  
RUN npm run build  
  
FROM node:20-alpine  
WORKDIR /usr/src/app  
COPY package*.json ./  
RUN npm ci --production --silent  
COPY --from=builder /usr/src/app/dist ./dist  
EXPOSE 4000  
CMD ["node", "dist/main.js"]
```

back/.dockerignore

```
node_modules  
dist  
npm-debug.log  
.env
```

back/src/main.ts

```
import { ValidationPipe } from '@nestjs/common';  
import { NestFactory } from '@nestjs/core';  
import { AppModule } from './app.module';  
  
async function bootstrap() {  
  const app = await NestFactory.create(AppModule);  
  app.enableCors();  
  app.useGlobalPipes(new ValidationPipe({ whitelist: true }));  
  const port = process.env.PORT ? parseInt(process.env.PORT, 10) : 4000;  
  await app.listen(port, '0.0.0.0');  
  console.log(`Backend listening on ${port}`);  
}  
bootstrap();
```

back/src/app.module.ts

```
import { Module } from '@nestjs/common';
import { MongooseModule } from '@nestjs/mongoose';
import { NewsModule } from '../news/news.module';

@Module({
  imports: [
    MongooseModule.forRoot(process.env.MONGO_URI || 'mongodb://mongo:27017/newsdb'),
    NewsModule,
  ],
})
export class AppModule {}
```

back/src/news/news.schema.ts

```
import { Prop, Schema, SchemaFactory } from '@nestjs/mongoose';
import { Document } from 'mongoose';

@Schema({ timestamps: true })
export class News {
  @Prop({ required: true })
  title: string;

  @Prop({ required: true })
  content: string;

  @Prop()
  author?: string;
}

export type NewsDocument = News & Document;
export const NewsSchema = SchemaFactory.createForClass(News);
```

back/src/news/dto/create-news.dto.ts

```
import { IsNotEmpty, IsOptional, IsString } from 'class-validator';

export class CreateNewsDto {
  @IsNotEmpty()
  @IsString()
  title: string;
}
```

```

@IsNotEmpty()
@IsString()
content: string;

@IsOptional()
@IsString()
author?: string;
}

```

back/src/news/dto/update-news.dto.ts

```

import { PartialType } from '@nestjs/mapped-types';
import { CreateNewsDto } from './create-news.dto';

export class UpdateNewsDto extends PartialType(CreateNewsDto) {}

```

back/src/news/news.service.ts

```

import { Injectable, NotFoundException } from '@nestjs/common';
import { InjectModel } from '@nestjs/mongoose';
import { Model } from 'mongoose';
import Redis from 'ioredis';
import { CreateNewsDto } from '../dto/create-news.dto';
import { UpdateNewsDto } from '../dto/update-news.dto';
import { News, NewsDocument } from '../news.schema';

@Injectable()
export class NewsService {
  private redis: Redis;

  constructor(@InjectModel(News.name) private newsModel: Model<NewsDocument>) {
    this.redis = new Redis({
      host: process.env.REDIS_HOST || 'redis',
      port: Number(process.env.REDIS_PORT || 6379),
    });
  }

  private async invalidateCache() {
    try {
      await this.redis.del('news:all');
    } catch (e) {
      // ignore cache errors
      console.warn('Redis invalidate error', e.message || e);
    }
  }
}

```

```

async create(createDto: CreateNewsDto) {
  const created = new this.newsModel(createDto);
  const res = await created.save();
  await this.invalidateCache();
  return res;
}

async findAll() {
  try {
    const cached = await this.redis.get('news:all');
    if (cached) return JSON.parse(cached);
  } catch (e) {
    // ignore cache errors
  }

  const list = await this.newsModel.find().sort({ createdAt: -1 }).exec();

  try {
    await this.redis.set('news:all', JSON.stringify(list), 'EX', 60);
  } catch (e) {
    // ignore set errors
  }

  return list;
}

async findOne(id: string) {
  const item = await this.newsModel.findById(id).exec();
  if (!item) throw new NotFoundException('News not found');
  return item;
}

async update(id: string, dto: UpdateNewsDto) {
  const updated = await this.newsModel.findByIdAndUpdate(id, dto, { new:
true }).exec();
  if (!updated) throw new NotFoundException('News not found');
  await this.invalidateCache();
  return updated;
}

async remove(id: string) {
  const removed = await this.newsModel.findByIdAndDelete(id).exec();
  if (!removed) throw new NotFoundException('News not found');
  await this.invalidateCache();
  return { deleted: true };
}
}

```

back/src/news/news.controller.ts

```
import { Body, Controller, Delete, Get, Param, Post, Put } from '@nestjs/common';
import { CreateNewsDto } from '../dto/create-news.dto';
import { UpdateNewsDto } from '../dto/update-news.dto';
import { NewsService } from '../news.service';

@Controller('news')
export class NewsController {
  constructor(private readonly service: NewsService) {}

  @Post()
  create(@Body() dto: CreateNewsDto) {
    return this.service.create(dto);
  }

  @Get()
  findAll() {
    return this.service.findAll();
  }

  @Get('/:id')
  findOne(@Param('id') id: string) {
    return this.service.findOne(id);
  }

  @Put('/:id')
  update(@Param('id') id: string, @Body() dto: UpdateNewsDto) {
    return this.service.update(id, dto);
  }

  @Delete('/:id')
  remove(@Param('id') id: string) {
    return this.service.remove(id);
  }
}
```

back/src/news/news.module.ts

```
import { Module } from '@nestjs/common';
import { MongooseModule } from '@nestjs/mongoose';
import { News, NewsSchema } from '../news.schema';
import { NewsService } from '../news.service';
import { NewsController } from '../news.controller';
```

```
@Module({
  imports: [MongooseModule.forFeature([
    { name: News.name, schema: NewsSchema }
  ])],
  providers: [NewsService],
  controllers: [NewsController],
})
export class NewsModule {}
```

front/ (Next.js v15 App Router) - files

Minimal front-end that consumes the API. Uses `NEXT_PUBLIC_API_URL` from `.env` to target the backend service.

front/package.json

```
{
  "name": "sample-front",
  "version": "1.0.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start -p 3000"
  },
  "dependencies": {
    "next": "^15.0.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0"
  }
}
```

front/.env (example)

```
NEXT_PUBLIC_API_URL=http://back:4000
```

front/Dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci --silent || npm i --silent
```

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY . .
COPY --from=deps /app/node_modules ./node_modules
RUN npm run build

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/.next ./next
COPY --from=builder /app/public ./public
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./package.json
EXPOSE 3000
CMD ["npm", "start"]

```

front/tsconfig.json (optional — if you want TypeScript)

```

{
  "compilerOptions": {
    "target": "es2020",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve"
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx"],
  "exclude": ["node_modules"]
}

```

front/app/page.tsx

```

'use client';
import React from 'react';
import NewsList from '../components/NewsList';

export default function HomePage() {
  return (
    <main style={{ padding: 20 }}>
      <h1>News (sample)</h1>
    </main>
  );
}

```

```

    <p>
      Backend: {process.env.NEXT_PUBLIC_API_URL}
    </p>
    <NewsList />
  </main>
);
}

```

front/components/NewsList.tsx

```

'use client';
import React, { useEffect, useState } from 'react';

type NewsItem = {
  _id: string;
  title: string;
  content: string;
  author?: string;
  createdAt?: string;
};

export default function NewsList() {
  const [items, setItems] = useState<NewsItem[]>([]);
  const [loading, setLoading] = useState(false);

  const fetchList = async () => {
    setLoading(true);
    try {
      const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/news`);
      const data = await res.json();
      setItems(data);
    } catch (e) {
      console.error(e);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchList();
  }, []);

  return (
    <div>
      <div style={{ marginBottom: 12 }}>
        <a href="/news/create">Create news</a>

```



```

    </div>

    {loading ? (
      <div>Loading...</div>
    ) : (
      items.map((it) => (
        <article key={it._id} style={{ borderBottom: '1px solid #eee',
padding: '8px 0' }}>
          <h3>{it.title}</h3>
          <p>{it.content}</p>
          <small>By {it.author || 'anonymous'}  {new Date(it.createdAt ||
''.toLocaleString())}</small>
          <div style={{ marginTop: 8 }}>
            <a href={` /news/${it._id}`}>View</a>
          </div>
        </article>
      ))
    )}
  </div>
);
}

```

front/app/news/create/page.tsx

```

'use client';
import React, { useState } from 'react';

export default function CreatePage() {
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  const [author, setAuthor] = useState('');
  const [loading, setLoading] = useState(false);

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);
    try {
      const res = await fetch(`${process.env.NEXT_PUBLIC_API_URL}/news`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ title, content, author }),
      });
      if (res.ok) {
        alert('created');
        window.location.href = '/';
      } else {

```

```

        const err = await res.text();
        alert('error: ' + err);
    }
    catch (e) {
        alert('error');
    }
    finally {
        setLoading(false);
    }
};

return (
    <main style={{ padding: 20 }}>
        <h2>Create News</h2>
        <form onSubmit={submit} style={{ display: 'grid', gap: 8, maxWidth: 600 }}>
    >
        <input placeholder="Title" value={title} onChange={(e) =>
setTitle(e.target.value)} required />
        <textarea placeholder="Content" value={content} onChange={(e) =>
setContent(e.target.value)} rows={6} required />
        <input placeholder="Author" value={author} onChange={(e) =>
setAuthor(e.target.value)} />
        <button type="submit" disabled={loading}>{loading ? 'Saving...' :
'Save'}</button>
    </form>
    </main>
);
}

```

front/app/news/[id]/page.tsx (view + edit link)

```

'use client';
import React, { useEffect, useState } from 'react';
import { useParams } from 'next/navigation';

export default function NewsView() {
    const params = useParams();
    const id = params?.id;
    const [item, setItem] = useState<any>(null);

    useEffect(() => {
        if (!id) return;
        fetch(`${process.env.NEXT_PUBLIC_API_URL}/news/${id}`).then(r =>
r.json()).then(setItem);
    }, [id]);

    if (!item) return <div>Loading...</div>;
}

```

```

return (
  <main style={{ padding: 20 }}>
    <h2>{item.title}</h2>
    <p>{item.content}</p>
    <small>By {item.author || 'anonymous'}</small>
    <div style={{ marginTop: 12 }}>
      <a href={` /news/${id}/edit`} >Edit</a>
    </div>
  </main>
);
}

```

front/app/news/[id]/edit/page.tsx (simple edit form)

```

'use client';
import React, { useEffect, useState } from 'react';
import { useParams } from 'next/navigation';

export default function EditPage() {
  const params = useParams();
  const id = params?.id;
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');
  const [author, setAuthor] = useState('');

  useEffect(() => {
    if (!id) return;
    fetch(`${process.env.NEXT_PUBLIC_API_URL}/news/${id}`)
      .then(r => r.json())
      .then((data) => {
        setTitle(data.title);
        setContent(data.content);
        setAuthor(data.author || '');
      });
  }, [id]);

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    await fetch(`${process.env.NEXT_PUBLIC_API_URL}/news/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title, content, author }),
    });
    window.location.href = `/news/${id}`;
  };
}

```

```

const remove = async () => {
  if (!confirm('Delete?')) return;
  await fetch(`${process.env.NEXT_PUBLIC_API_URL}/news/${id}`, { method:
'DELETE' });
  window.location.href = '/';
};

return (
  <main style={{ padding: 20 }}>
    <h2>Edit</h2>
    <form onSubmit={submit} style={{ display: 'grid', gap: 8, maxWidth: 600 }}
  >
    <input value={title} onChange={(e) => setTitle(e.target.value)}
required />
    <textarea value={content} onChange={(e) => setContent(e.target.value)}
rows={6} required />
    <input value={author} onChange={(e) => setAuthor(e.target.value)} />
    <div style={{ display: 'flex', gap: 8 }}>
      <button type="submit">Save</button>
      <button type="button" onClick={remove}>Delete</button>
    </div>
  </form>
</main>
);
}

```

Additional notes & tips

- **Volumes:** Mongo and Redis use named volumes (persist data between restarts). The `docker-compose.yml` defines them; Swarm `docker-stack.yml` does too.
- **Environment:** `docker-compose` uses `env_file` entries to load `.env` for front/back. Edit `back/.env` and `front/.env` before starting if you want different values.
- **CORS:** The NestJS app enables CORS globally in `main.ts` so the front can call the API directly.
- **Cache:** Redis caching in `NewsService` only caches the list for 60 seconds and invalidates on create/update/delete — simple and clear.
- **Swarm images:** For a multi-node Swarm you typically push images to a registry (Docker Hub / private) and reference `yourrepo/sample_front:tag` in `docker-stack.yml`.

Troubleshooting

- If `front` can't reach `back` inside the cluster, ensure they are on the same network (they are in the compose examples) and that `NEXT_PUBLIC_API_URL` points to `http://back:4000` (service DNS name inside the network).
 - If using `docker stack deploy` and images are not found on worker nodes, push them to a registry or use a registry accessible from all nodes.
 - For host-based testing remember to add `/etc/hosts` entries.
-

Next steps I can help with

- Generate a downloadable ZIP of the entire project tree.
 - Convert the frontend to use server-side fetching with Next.js App Router server components (if you prefer that pattern).
 - Add authentication (JWT) to the back and a login UI to the front.
-

End of document.